

Analisis completo del sistema

Arquitectura, modelo de datos, seguridad, flujos operativos y riesgos

Campo	Detalle
Sistema	Ticket Tracking System
Documento	Analisis tecnico
Fecha	1 de julio de 2026
Version	1.0

1. Resumen ejecutivo

El sistema Ticket Tracking System es una mesa de ayuda web para registrar solicitudes e incidencias, convertir casos en tickets operativos, asignarlos a tecnicos, controlar SLA, auditar cambios y administrar usuarios, sedes y ubicaciones.

La aplicacion combina una API Java con Javalin, una base PostgreSQL y un frontend estatico servido por Nginx. El despliegue recomendado para el equipo es Docker Compose, que levanta API, web, PostgreSQL y Adminer en una sola orden.

Estado funcional: Los flujos modernos usan /service-cases, /tickets, /admin/users, /sites y /locations. Las rutas legacy relacionadas con /user/requests, /client/requests y /technician/requests responden 410 Gone para evitar el camino antiguo con SQL concatenado.

2. Objetivo y alcance

Area	Descripcion
Registro de casos	Usuarios y administradores crean casos tipo REQUEST o INCIDENT con prioridad, sede, ubicacion, titulo y descripcion.
Operacion tecnica	Tecnicos y administradores consultan colas, crean tickets, asignan responsables, agregan comentarios, cambian estado y resuelven tickets.
Gobierno	Administradores gestionan usuarios, roles operativos, sedes y ubicaciones.
Seguimiento	El sistema expone dashboard por tipo de caso, SLA, tickets abiertos, en proceso y resueltos.
Notificaciones	Se guardan notificaciones por email con reintentos. El modo por defecto registra en log; SMTP se configura por variables de entorno.

3. Arquitectura general

Componente	Tecnologia	Responsabilidad
Frontend	HTML, CSS, JavaScript	login.html y employee.html consumen la API, muestran vistas por rol, formularios y bandejas.
API	Java 8, Javalin 4.5	Expone endpoints REST, valida permisos, ejecuta reglas de negocio y serializa JSON.
Base de datos	PostgreSQL 14	Persiste usuarios, sesiones, casos, tickets, SLA, auditoria, notificaciones, adjuntos y catalogos.
Servidor web	Nginx	Sirve archivos estaticos del frontend en el puerto 8080.
Adminer	Adminer 4.8	Herramienta web para inspeccionar la base durante desarrollo.
Docker	Docker Compose	Orquesta servicios locales y variables de entorno.

3.1 Flujo de ejecucion

Paso	Descripcion
1	El usuario abre http://localhost:8080/login.html.
2	El frontend envia credenciales a POST /login en la API.
3	AuthService valida usuario, hash de password, estado activo y genera token de sesion.
4	El frontend guarda el token en sessionStorage y lo envia como Bearer token.
5	AuthMiddleware autentica y AccessPolicy autoriza cada endpoint segun rol y permiso.
6	Los servicios consultan PostgreSQL con PreparedStatement y devuelven JSON.

4. Roles y permisos

Rol	Capacidades principales
USER	Crear casos, ver sus casos, ver tickets vinculados, actualizar perfil, cerrar sesion y usar metadatos de sedes/ubicaciones.
TECH	Ver dashboard operativo, colas, casos asignados, crear tickets desde casos abiertos, asignar, actualizar y resolver tickets.
ADMIN	Incluye capacidades de USER y TECH, mas gestion de usuarios, sedes, ubicaciones, reintento de notificaciones y edicion protegida de tickets/casos.

5. Modelo de datos

Tabla	Proposito	Relaciones clave
roles	Catalogo USER, TECH, ADMIN.	app_users.role_id
app_users	Usuarios modernos y datos de contacto.	roles, auth_sessions, service_case, ticket
auth_sessions	Sesiones por token hash y vencimiento.	app_users
password_reset_tokens	Tokens temporales para cambio de password.	app_users
sites	Sedes institucionales.	service_case, locations
locations	Aulas, laboratorios y areas por sede.	sites, service_case
sla_policies	Políticas de tiempos por tipo y prioridad.	service_case, ticket
service_case	Caso base REQUEST o INCIDENT.	app_users, sites, locations, ticket
ticket	Trabajo operativo asociado a un caso.	service_case, app_users, ticket_updates

Tabla	Proposito	Relaciones clave
ticket_updates	Comentarios y cambios de estado/asignacion.	ticket, app_users
attachments	Metadatos de adjuntos para caso, ticket o actualizacion.	service_case, ticket, ticket_updates
notifications	Notificaciones in-app/email y estado de envio.	app_users, service_case, ticket
audit_logs	Registro historico de acciones y cambios.	actor_user_id, entity_type, entity_id

5.1 Compatibilidad legacy

La base conserva tablas antiguas employee_type, employees, ticket_requests y tickets. La migracion crea app_users, service_case y ticket a partir de esos datos para no perder informacion historica. La API moderna no debe depender de los endpoints legacy retirados.

6. Endpoints principales

Modulo	Endpoint	Uso
Salud	GET /health	Verifica que la API este arriba. Publico.
Autenticacion	POST /login, POST /logout, GET /me	Inicio, cierre y perfil autenticado.
Password	POST /password/forgot, POST /password/reset	Flujo local de recuperacion.
Usuarios	GET/POST /admin/users, PATCH /admin/users/{id}	Gestion administrada y paginada.
Catalogos	GET/POST/PATCH /sites, /locations	Sedes y ubicaciones.
Casos	POST/GET /service-cases, GET /service-cases/{id}	Registro y consulta de casos.
Colas	GET /queues/service-cases	Casos abiertos sin ticket.
Tickets	GET/PATCH /tickets, POST /tickets/{id}/assign, POST /tickets/{id}/resolve	Operacion tecnica.
Auditoria	GET /tickets/{id}/audit	Historial de cambios visible en detalle.
Notificaciones	POST /notifications/retry	Reintento administrativo de emails pendientes.

6.1 Paginacion

Los listados modernos /service-cases, /tickets y /admin/users aceptan page y pageSize. La respuesta es un objeto con data y pagination. page inicia en 1; pageSize usa 50 por defecto y maximo 100.

7. Reglas de negocio relevantes

Regla	Detalle
Creacion de caso	Requiere type REQUEST/INCIDENT, titulo de 8 a 160 caracteres, descripcion de 20 a 4000, prioridad y sede activa.
SLA	Se selecciona la politica activa por tipo/prioridad; si no existe, SlaEngine usa defaults: REQUEST 72h e INCIDENT 24h.
Cola sin ticket	Solo casos OPEN sin ticket activo aparecen en colas para TECH/ADMIN.
Crear ticket	Valida tecnico activo, caso OPEN y que el caso no tenga ticket previo.
Actualizar ticket	TECH puede cambiar estado OPEN/IN_PROGRESS, prioridad y comentarios; ADMIN puede editar campos protegidos.
Resolver	Requiere resolucion minima de 12 caracteres; actualiza ticket y caso a RESOLVED y registra update publico.
Auditoria	Cambios relevantes insertan audit_logs con before_data y after_data JSONB.

8. Seguridad

La autenticacion se basa en token aleatorio almacenado como SHA-256 en auth_sessions. La API acepta Authorization: Bearer, X-Auth-Token o token en body JSON para compatibilidad. Las sesiones duran 480 minutos.

PasswordHasher soporta hashes modernos PBKDF2 y migra automaticamente passwords legacy marcados como legacy: cuando el usuario inicia sesion correctamente.

AuthMiddleware aplica AccessPolicy antes de cada request, salvo rutas publicas. Las denegaciones pueden auditarse con motivo NO_POLICY_MATCH o MISSING_PERMISSION.

9. Despliegue y operacion local

Elemento	Valor recomendado
Web	http://localhost:8080/login.html
API	http://localhost:8081
Health	http://localhost:8081/health
Adminer	http://localhost:8082
PostgreSQL host	localhost:5432
Docker Compose	docker compose up --build
Reset de datos	docker compose down -v; docker compose up --build

10. Pruebas y CI

El workflow Java CI valida Maven, compila y ejecuta pruebas clave que no dependen de una base externa ni de drivers de navegador: EntityTests, AuthorizationPolicyTests, SlaEngineTests, DashboardMetricsTests y PaginationTests.

DataAccessTests, ServiceLayerTests y Cucumber/Selenium quedan fuera del CI ligero porque dependen de PostgreSQL externo o drivers UI. Para pruebas integrales se recomienda ejecutarlas con Docker y datos controlados.

11. Riesgos y mejoras pendientes

Riesgo	Impacto	Recomendacion
Frontend con API URL fija	Si el puerto 8081 cambia, el HTML debe ajustarse.	Externalizar API_BASE_URL o proxy inverso.
Adjuntos sin UI/API completa	La base esta lista, pero falta carga/descarga de binarios.	Agregar endpoints multipart y storage local/S3.
Pruebas integrales limitadas	CI no valida todo el flujo end-to-end.	Agregar ambiente de test con PostgreSQL en GitHub Actions.
Drivers UI eliminados	Selenium local requiere instalacion externa o WebDriver manager.	Actualizar runner UI a drivers gestionados.
Credenciales demo	Usuarios pass/pass no sirven para produccion.	Cambiar secretos y password iniciales antes de despliegue real.